Frontend Integration Guide - Step 1: Requirements

This document provides a technical roadmap for the frontend developer to implement **Step 1 (Requirements Generation)** of the CHB Recruitment Portal.

Overview

Step 1 is the foundation of the recruitment process. Its purpose is to define the "Gap" (how many faculty members are needed) by comparing current student intake against regulatory norms (DTE/AICTE).

Key Components

1. Institution & Course Management

The system requires a primary setup of institutions and their specific courses.

- **Primary UI Elements:**
  - Institution Profile (Name, DTE Code, District, Type).
  - Course List (Engineering Diploma, Engineering Degree, Pharmacy, etc.).
- **Backend Prefix:** `/api/requirements`
- **Key Endpoints:**
  - `POST /api/requirements/institutions`: Create a new institution.
  - `GET /api/requirements/institutions`: List all institutions.
  - `POST /api/requirements/courses`: Add a course to an institution.

2. Intake & Norm Definition (The "Course Setup")

For every course in an academic year, the Principal must define the student intake and the applicable norms.

- **Data to Capture:**
  - **Intake:** Approved Intake (e.g., 60), Actual Admitted (e.g., 58).
  - **Norms:** Faculty-to-Student Ratio (e.g., 1:20), Minimum Qualification (e.g., M.E./M.Tech), Workload (18 hours/week).
- **Workflow:**
  - The UI should provide a "Course Setup" form.
  - **Endpoint:** `POST /api/requirements/course-setup`
  - **Request Body:**

```
{
  "institution_id": 1,
  "course_name": "Computer Engineering",
  "academic_year": "2026-2027",
  "approved_seats": 60,
  "actual_admitted": 58,
  "faculty_student_ratio": 20.0,
  "min_qualification": "M.E. in Computer Engineering",
```

```
    "grade_requirement": "First Class"
}
```

3. Faculty Requirement Calculator (AI-Assisted)

Once intake and norms are saved, the system uses a rule-based engine and AI validation to determine staffing needs.

- **How it works:**
  - **Calculation:** Required Faculty = `max(Approved Intake, Actual Admitted) / Faculty-Student Ratio` (rounded up).
  - **Historical Comparison:** The system automatically fetches data from the previous academic year to check for sudden jumps or drops in requirements.
  - **Anomaly Flagging:** It flags variations (e.g., if required faculty increases by 50% while student intake only increased by 5%, it triggers a 'HIGH' severity anomaly).
- **Outcome:** The AI generates a **"Suggested Requirement Summary"** which includes the computed number and a qualitative analysis.
- **Workflow:**
  1. **Calculate:** Call `POST /api/requirements/generate`.
  2. **Validate:** Call `POST /api/requirements/validate`.
- **Note:** Final approval of these requirements remains with the Directorate; the AI is a decision-support tool.

What to add in the Frontend (UI Checklist)

The frontend developer should implement the following features for Step 1:

1. Requirements Dashboard

- **Status Badges:** Display the current state of each course (e.g., `SETUP_PENDING`, `CALCULATED`, `AI_FLAGGED`, `VALIDATED`).
- **Quick Actions:** Buttons for "Setup Norms", "Calculate", and "AI Audit".

2. The Calculator Component

- **Interactive Form:** Fields for Approved Intake, Actual Admitted, and Norms (Ratio, Qualification).
- **Real-time Preview:** If possible, show a "pre-calculated" number on the UI as the user types, though the final number comes from the backend.
- **Historical Context:** A small card or tooltip showing "Last Year's Requirement: X" to help the user identify typos.

3. AI Insights Panel

- **Validation Results:** When the user clicks "Validate with AI", show a panel with:
  - **Analysis Summary:** A natural language explanation from the AI.
  - **Anomaly List:** A list of flagged items (e.g., "Warning: Student intake dropped, but requested faculty remains same").
  - **Confidence Score:** A visual meter showing the AI's confidence in the data.

4. Admin Overrides

- **Remarks Field:** Allow the Principal to add remarks if they want to override or justify an anomaly flagged by the AI.

Frontend Tips

- **Standard Envelope:** All API responses follow the format: `{"status": "success", "message": "...", "data": {...}}`.
- **Error Handling:** If validation fails, the backend returns a `422` or `400` with a clear message. Always display `error.response.data.message` to the user.
- **Academic Year:** Ensure the academic year is consistently sent as a string (e.g., "2026-2027").

Postman Reference

Refer to the **Requirements (Step 1)** folder in the `CHB_Portal.postman_collection.json` for live examples of every request and response.